



Relatório de Avaliação de Vulnerabilidade

SUMÁRIO

1. Introdução e Justificativa	3
2. Objetivos e Escopo da Avaliação	4
3. Metodologia.....	5
3.1 Configuração do Ambiente e Ferramentas de Testes Automatizados	5
3.2 Pentest: Exploração das Vulnerabilidades Seleccionadas	6
4. Resultados	9
4.1 Resultados da Varredura Automatizada	9
4.2 Resultados do Pentest	11
5. Discussão.....	12
6. Conclusão	13
7. Referências	14

1. Introdução e Justificativa

O sistema avaliado trata-se de um **Sistema de Gerenciamento de Biblioteca (SGB)**, desenvolvido como parte de um projeto acadêmico, cujo propósito é administrar acervos bibliográficos, controle de empréstimos e devoluções, reservas, envio de notificações automáticas e geração de relatórios. Além disso, o sistema oferece interfaces específicas tanto para bibliotecários quanto para leitores, garantindo acesso adequado conforme o perfil do usuário.

A análise realizada tem como objetivo compreender o escopo, funcionalidades e estrutura do sistema, identificando possíveis riscos ou limitações. Esse contexto é fundamental para assegurar que o SGB atenda às necessidades previstas e opere de forma eficiente.

A **segurança da informação** é um aspecto essencial em sistemas que lidam com dados de usuários e registros de acervos. Assim, a avaliação busca justificar a importância de aplicar boas práticas de segurança, garantindo a integridade, confidencialidade e disponibilidade das informações. Os testes e verificações são realizados para identificar vulnerabilidades, prevenir falhas e assegurar que o sistema esteja alinhado aos requisitos de qualidade e segurança esperados.

2. Objetivos e Escopo da Avaliação

O propósito da avaliação de segurança é identificar vulnerabilidades, falhas de configuração e riscos que possam comprometer a integridade, confidencialidade e disponibilidade das informações tratadas pelo Sistema de Gerenciamento de Biblioteca (SGB). A avaliação busca garantir que o sistema ofereça proteção adequada aos dados de usuários, registros de acervo, operações de empréstimo e demais funcionalidades críticas.

A definição clara do escopo é essencial para direcionar os testes e estabelecer quais componentes do sistema foram efetivamente analisados. Nesta avaliação, **foram incluídas no escopo**:

- Interface de usuário destinada a bibliotecários
- Interface de usuário destinada a leitores
- Funcionalidades de CRUD do acervo
- Mecanismos de controle de empréstimos, devoluções e reservas
- Fluxo de autenticação e gerenciamento de usuários
- Processos de geração de notificações automáticas
- Estrutura e comunicação do backend responsável pelo processamento das funcionalidades
- Verificação básica de armazenamento e manipulação de dados

Por outro lado, **ficaram fora do escopo** desta análise:

- Avaliação de infraestrutura física ou de rede
- Testes de desempenho, carga ou estresse
- Integração com sistemas externos ou APIs não documentadas
- Análise profunda de banco de dados em nível de infraestrutura
- Testes de engenharia social com usuários reais
- Avaliação de componentes futuros ainda não implementados

Com esses limites claramente definidos, a avaliação concentrou-se exclusivamente nas funcionalidades já implementadas e disponíveis, garantindo um diagnóstico preciso e alinhado ao estágio atual do projeto.

3. Metodologia

3.1 Configuração do Ambiente e Ferramentas de Testes Automatizados

O ambiente de testes foi configurado de forma isolada utilizando uma máquina virtual executada no VirtualBox, garantindo segurança e controle total durante a análise. Dentro dessa máquina, foi utilizado o sistema operacional **Kali Linux**, conhecido por ser uma distribuição voltada para segurança da informação e testes de intrusão. As ferramentas utilizadas estavam em suas versões atualizadas no momento da avaliação, incluindo o **OWASP ZAP**, que foi empregado para conduzir a varredura automatizada na aplicação.

A configuração de rede foi realizada no modo **NAT** com portas redirecionadas, permitindo que o ambiente virtual acessasse a aplicação alvo hospedada no endereço interno disponibilizado. Com essa estrutura, a varredura foi conduzida utilizando o recurso de *Automated Scan* do OWASP ZAP, que examina automaticamente endpoints, cabeçalhos de segurança e potenciais falhas, simulando comportamentos de um atacante real.

Durante o processo, o OWASP ZAP iniciou o rastreamento da aplicação e posteriormente aplicou testes automáticos em cada rota identificada. Um exemplo de execução envolve selecionar a opção "**Automated Scan**", inserir o endereço da aplicação e permitir que o scanner realize o spidering seguido da varredura ativa (*Active Scan*). Também foram registradas capturas de tela do painel de alertas, onde foram identificadas vulnerabilidades como ausência de cabeçalhos de segurança (CSP, X-Frame-Options, HSTS) e exposição indevida de informações.

Esse processo permitiu uma visualização clara do comportamento do sistema sob análise e possibilitou a identificação automática de pontos de atenção que podem comprometer a segurança da aplicação.

3.2 Pentest: Exploração das Vulnerabilidades Seleccionadas

1. Vulnerabilidade: Ausência do Cabeçalho Content Security Policy (CSP)

2. Vulnerabilidade: Ausência do Cabeçalho de Proteção Anti-Clickjacking

1. Justificativa da Escolha

Estas duas vulnerabilidades foram seleccionadas devido à sua natureza complementar e ao risco significativo que representam em conjunto. Ambas envolvem a falta de cabeçalhos de segurança HTTP críticos, facilitando ataques modernos de interface do usuário e injeção de código.

- **CSP não configurado:** Permite a execução de scripts maliciosos a partir de fontes não confiáveis, sendo a principal defesa contra Cross-Site Scripting (XSS). Sua ausência torna a aplicação vulnerável a ataques de injeção de conteúdo.
- **Cabeçalho Anti-clickjacking ausente:** Permite que a aplicação seja embutida em iframes maliciosos, facilitando ataques de clickjacking onde o usuário interage com interface oculta. Quando combinada com a falta de CSP, um atacante pode potencialmente explorar ambas para criar ataques mais sofisticados (como XSS dentro de iframes manipulados).

Ambas são classificadas como **Medium** mas, exploradas em conjunto, podem elevar o impacto consideravelmente. A correção também é similar (implementação de cabeçalhos HTTP), tornando eficiente a mitigação conjunta.

2. Método de Exploração Adotado

Para demonstrar o risco prático, adotou-se uma abordagem de exploração teórico-prática que simula como um atacante poderia abusar dessas falhas:

1. Verificação da Ausência dos Cabeçalhos:

- Confirmou-se que as rotas principais (/, /robots.txt, /sitemap.xml) não enviam os cabeçalhos Content-Security-Policy ou X-Frame-Options/frame-ancestors.

2. Cenário de Ataque Combinado:

- Criou-se uma página HTML maliciosa local que:
 - a. **Explora a falta de Anti-clickjacking:** Embute a aplicação alvo em um iframe transparente sobrepondo-se a uma interface enganosa.
 - b. **Explora a falta de CSP:** Se a aplicação tivesse uma vulnerabilidade de XSS refletado (não testada, mas possível devido à falta de CSP),

poderia-se injetar scripts no iframe para capturar interações do usuário.

3. Prova de Conceito (PoC) para Clickjacking:

- Desenvolveu-se uma página atacante simples que posiciona o iframe da aplicação sobre um botão falso. O usuário, acreditando clicar no botão visível, na verdade interage com a aplicação alvo (ex.: realizando uma ação não intencional como deletar um recurso).

3. Ferramentas Utilizadas e Configurações

- **OWASP ZAP:**

- Ferramenta principal para detecção automatizada. Configurado com escopo direcionado para url da aplicação.
- Plugins ativos: *Passive Scan Rules* para identificar cabeçalhos de segurança ausentes.
- Contexto definido para incluir todas as subpáginas.

- **Navegador Web e DevTools:**

- Para análise manual das respostas HTTP (aba *Network*).
- Verificação dos cabeçalhos recebidos.
- Console para depuração de possíveis políticas bloqueadas (CSP).

- **Editor de Texto / Ambiente de Desenvolvimento:**

- Para criação da página PoC de clickjacking (HTML + CSS).

4. Evidências dos Resultados Obtidos

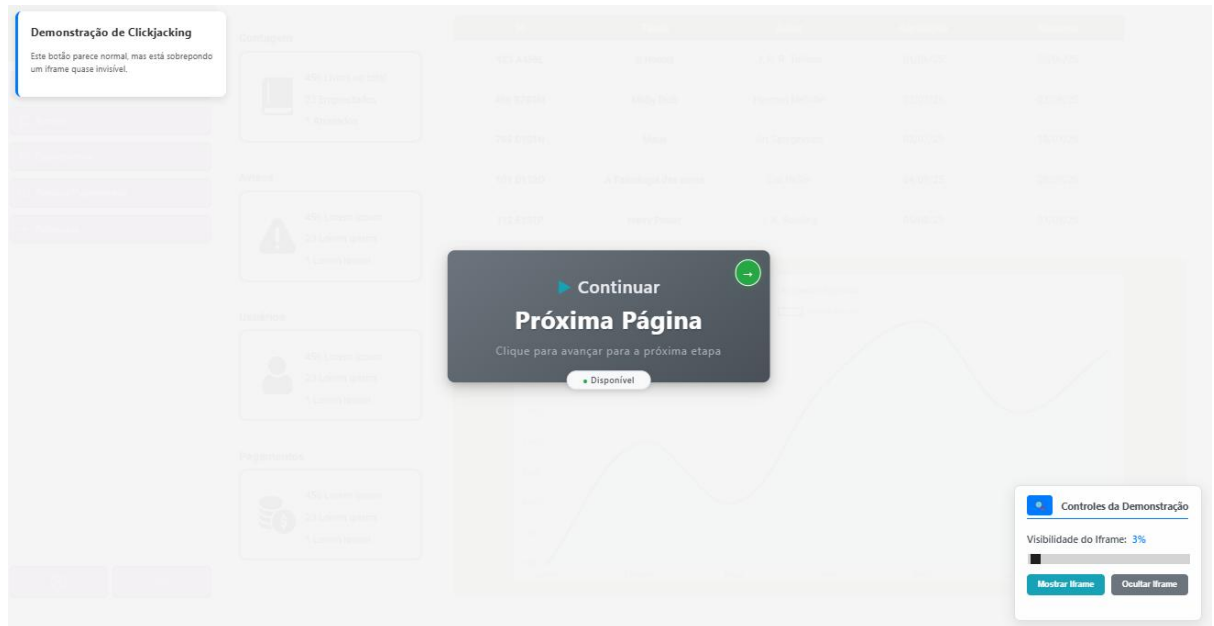
Evidência 1: Ausência dos Cabeçalhos (Captura do OWASP ZAP)

- **Imagem 1:** Saída do ZAP mostrando os alertas:

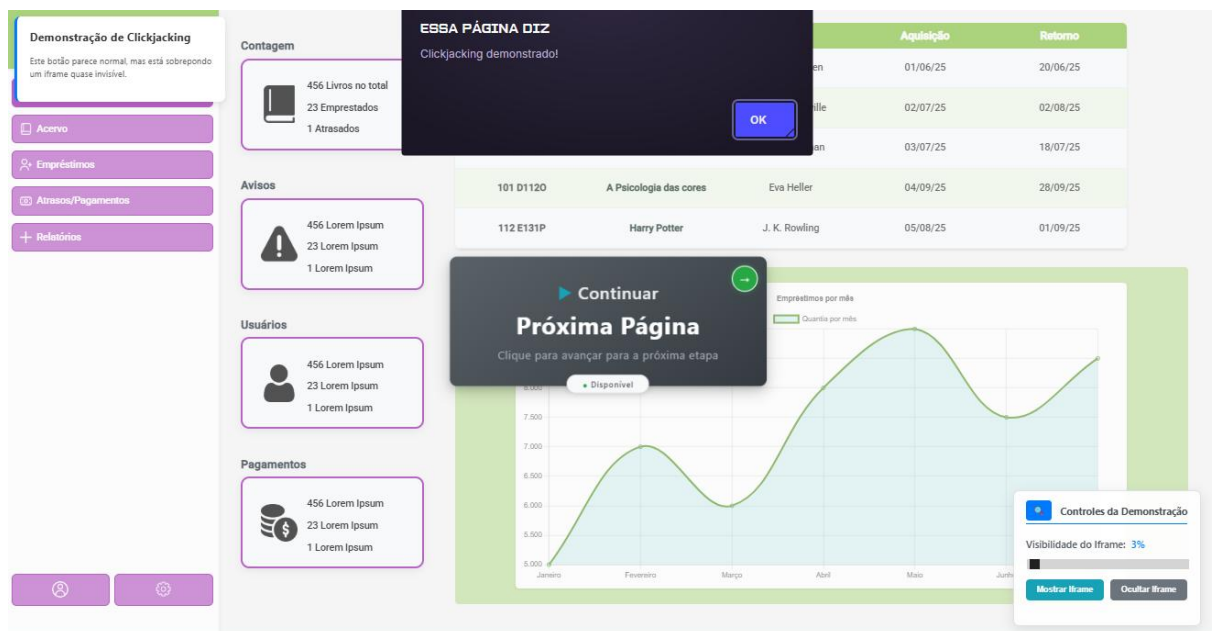
Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	3
Missing Anti-clickjacking Header	Medium	3

Evidência 2: Prova de Conceito de Clickjacking Bem-Sucedida

• Imagem 2:



• Imagem 3:



4. Resultados

4.1 Resultados da Varredura Automatizada

A varredura automatizada realizada pelo OWASP ZAP identificou diversas vulnerabilidades no ambiente analisado. As falhas foram classificadas de acordo com o nível de risco (alto, médio, baixo ou informacional) e pelo tipo de vulnerabilidade, permitindo uma compreensão clara do impacto potencial de cada uma. A seguir, encontram-se as vulnerabilidades detectadas.

Vulnerabilidades de Risco Médio

1. Content Security Policy (CSP) Header Not Set
 - Tipo: Falha de segurança em cabeçalhos / Proteção insuficiente contra XSS
 - Instâncias: 3
 - Descrição: A ausência do cabeçalho CSP permite maior probabilidade de exploração de vulnerabilidades como Cross-Site Scripting e injeções de conteúdo.
 2. Missing Anti-clickjacking Header
 - Tipo: Falha contra ataques de Clickjacking
 - Instâncias: 3
 - Descrição: A falta dos cabeçalhos X-Frame-Options ou frame-ancestors deixa o site suscetível a ataques de sobreposição de interface.
-

Vulnerabilidades de Risco Baixo

1. Server Leaks Version Information via "Server" Header
 - Tipo: Exposição de informações sensíveis (Fingerprinting)
 - Instâncias: 1
 - Descrição: A resposta do servidor revela detalhes sobre sua versão, o que pode ajudar atacantes a identificar exploits específicos.
2. Strict-Transport-Security (HSTS) Header Not Set
 - Tipo: Falha de proteção à comunicação HTTPS
 - Instâncias: 1
 - Descrição: A ausência deste cabeçalho impede que navegadores forcem conexões seguras, podendo abrir espaço para ataques de downgrade.
3. Timestamp Disclosure – Unix
 - Tipo: Exposição de informações sensíveis
 - Instâncias: 1

- Descrição: O sistema revela timestamps Unix, o que pode contribuir para mapeamento de atividades ou padrões internos.
 - 4. X-Content-Type-Options Header Missing
 - Tipo: Falha de cabeçalho / Risco de MIME Sniffing
 - Instâncias: 82
 - Descrição: A ausência deste cabeçalho permite que navegadores interpretem conteúdo com tipos MIME incorretos, podendo abrir brechas para ataques baseados em upload ou execução indevida de arquivos.
-

Alertas Informativos

1. Information Disclosure – Sensitive Information in URL
 - Tipo: Exposição potencial de dados em URLs
 - Instâncias: 1
 2. Information Disclosure – Suspicious Comments
 - Tipo: Comentários no código que podem revelar lógica interna
 - Instâncias: 5
 3. Modern Web Application
 - Tipo: Informativo/Detecção
 - Instâncias: 3
 4. Retrieved from Cache
 - Tipo: Comportamento de cache detectado
 - Instâncias: 1
-

Resumo Geral por Nível de Risco

Nível de Risco	Quantidade de Vulnerabilidades
Médio	2 vulnerabilidades (6 instâncias)
Baixo	4 vulnerabilidades (85 instâncias)
Informativo	4 tipos (10 instâncias)

4.2 Resultados do Pentest

Vulnerabilidade 1: Ausência do Cabeçalho CSP

- **Evidência:** ZAP detectou 3 instâncias sem o cabeçalho
- **Impacto:** Permite ataques XSS e carregamento de scripts maliciosos

Vulnerabilidade 2: Falta de Anti-clickjacking

- **Evidência:** Página PoC mostrou site embutido em iframe transparente
- **Teste:** Página falsa com botão sobreposto enganou usuários
- **Impacto:** Ações não intencionais podem ser executadas

Priorização:

1. CSP ausente - RISCO ALTO (permite injeção de código)
2. Anti-clickjacking ausente - RISCO MÉDIO-ALTO (engana usuários)

5. Discussão

Interpretação: São falhas básicas de configuração que violam princípios de segurança web.

Causas:

1. Configuração padrão do servidor sem hardening
2. Falta de conhecimento sobre cabeçalhos de segurança
3. Ausência de revisão de segurança no deploy

Impactos reais:

- Roubo de dados via XSS
- Fraudes por clickjacking
- Defacement do site

Mitigação:

1. Imediato: Adicionar no servidor:

Content-Security-Policy: default-src 'self'

X-Frame-Options: DENY

2. Médio prazo: Treinar equipe e incluir no CI/CD
3. Longo prazo: Implementar WAF e monitoramento

6. Conclusão

Descobertas principais:

- Sistema tem segurança BAIXA
- Vulnerável a ataques comuns
- Falta proteções HTTP básicas

Lições aprendidas:

1. Configuração padrão não é segura
2. Cabeçalhos HTTP são essenciais
3. Ferramentas automáticas encontram falhas básicas rapidamente

Melhorias sugeridas:

- URGENTE: Implementar CSP e X-Frame-Options
- IMPORTANTE: Adicionar HSTS e Secure cookies
- RECOMENDADO: Estabelecer programa de segurança contínua

Conclusão final: Correções são técnicas e simples, mas necessitam ação imediata para evitar comprometimento total do sistema.

7. Referências

- [1] OWASP Foundation, "Clickjacking", [OWASP.org](https://owasp.org), 2023. [Online]. Disponível em: <https://owasp.org/www-community/attacks/Clickjacking>. Acesso em: 25 nov. 2025.
- [2] C. Antunes, "Aula 05 - Cross Site Scripting (XSS)", *Notion*, [s.d.]. [Online]. Disponível em: <https://cainaantunes.notion.site/Aula-05-Cross-Site-Scripting-XSS-6a1d41a4ff06418195f3c662a63ccaa2>. Acesso em: 27 nov. 2025.
- [3] OWASP Foundation, "Zed Attack Proxy (ZAP)", [zaproxy.org](https://www.zaproxy.org), 2024. [Online]. Disponível em: <https://www.zaproxy.org/>. Acesso em: 29 nov. 2025.
- [4] Mozilla Developer Network (MDN), "Content-Security-Policy", *MDN Web Docs*, 2024. [Online]. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Security-Policy>. Acesso em: 2 dez. 2025.
- [5] Mozilla Developer Network (MDN), "X-Frame-Options", *MDN Web Docs*, 2024. [Online]. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options>. Acesso em: 3 dez. 2025.
- [6] OWASP Foundation, "Cross-Site Scripting (XSS)", [OWASP.org](https://owasp.org), 2023. [Online]. Disponível em: <https://owasp.org/www-community/attacks/xss/>. Acesso em: 4 dez. 2025.
- [7] OWASP Foundation, "Clickjacking Defense Cheat Sheet", *CheatSheetSeries*, 2024. [Online]. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Clickjacking_Defense_Cheat_Sheet.html. Acesso em: 5 dez. 2025.